

Санкт-Петербургский государственный университет
Кафедра системного программирования

Анализ подходов к разработке VPU и архитектуры ядра scr1

Бикеев Кирилл Алексеевич, группа 25.M71-мм

Научный руководитель: ст. преподаватель кафедры ИАС, К. К. Смирнов

Санкт-Петербург
2026

- Архитектура RISC-V набирает популярность за счёт открытости и гибкости; в альянс RISC-V входят Google, nvidia, Huawei, Qualcomm
- Компания Syntacore разработала линейку RISC-V-ядер; младшее из них — scr1 — предмет исследования (широкое применение, открытая архитектура)
- scr1 — ядро MCU-класса, оптимизированное под площадь и энергопотребление, поэтому в текущей реализации **нет BPU** (Branch Prediction Unit)
- **Вопрос:** какой подход к реализации BPU даёт оптимальное соотношение прироста производительности к приросту энергопотребления и площади?

Цель работы — изучение подходов к реализации BPU и архитектуры scr1.

Задачи:

- сделать обзор подходов к реализации BPU
- сделать обзор существующих open-source RISC-V-совместимых архитектур с точки зрения реализации и внедрения BPU
- предложить наиболее оптимальные подходы для scr1
- реализовать дамп счётчиков scr1

Решение фиксировано на этапе проектирования, не зависит от истории выполнения.

- «always not taken» / «always taken» — всегда предсказывать одно направление; статистически ветвь берётся в $\sim 60\%$ случаев, поэтому «always taken» эффективнее.

В scr1 сейчас используется «always not taken»

- BTFNT (backwards taken / forwards not-taken) — обратные переходы (обычно циклы) предсказываются как взятые, прямые — как невзятые

+ простота, нулевая аппаратная цена — не адаптируются к поведению программы

Решение принимается во время работы по накопленной истории переходов (ценой площади, энергопотребления и критического пути).

- 2-битные насыщающие счётчики, индексируемые по РС: гистерезис гасит единичные аномалии (важно для циклов); точность $\sim 80\text{--}90\%$, но без учёта корреляции переходов
- Двухуровневые адаптивные схемы: регистр истории (BHR) + таблица образцов (PHT). История/таблица бывают глобальными, индивидуальными, групповыми — семейство GAg, PAp, SAg, ...
- gselect / gshare (McFarling): объединяют РС и историю; в gshare — операцией XOR. gshare — де-факто эталонная базовая схема

ВРU в открытых RISC-V-ядрах МСU-класса

Ядро	Конвейер	ВРU
scr1	2–4 ст.	нет
lbex	2 ст.	опц. статический SBP (BTFNT) + BT-ALU
CV32E40P	4 ст.	предсказание в стадии выборки
VexRiscv	настраиваемый	динамический BTB + 2-битный счётчик
CVA6	6 ст.	BHT + BTB + RAS (прикладной класс)

- В МСU-классе ВРU либо отсутствует, либо это статический / простой динамический предсказатель
- Полноценные многотабличные схемы появляются лишь в прикладном классе с более глубоким конвейером (CVA6)

Количественные ориентиры и выбор для scr1

- Доля мощности BPU: предсказатель направления $< 1\%$, блок в целом (с BTV) $\sim 7\text{--}10\%$ энергопотребления процессора
- Во встраиваемых ядрах под предсказатель выделяется ≤ 32 Кбит RAM — приходится искать компромисс ёмкость/точность
- Design-space TAGE: area-first даёт $-10,44\%$ площади и $-11,6\%$ мощности ценой $-1,89\%$ точности; TAGE 0,25 КБ $\approx$ gshare 4 КБ

Вывод для scr1: лёгкий статический предсказатель либо минимальная bimodal/gshare; полноценный BPU с BTV и TAGE нецелесообразен без пересмотра бюджета площади ядра.

Эмпирическая оценка BPU невозможна без измерительной инфраструктуры — поэтому на текущем этапе реализован дампы аппаратных счётчиков.

- Источник данных — CSR-счётчики RISC-V: `cycle`, `instret`, `time` (64-битные, на 32-битном ядре — пары lo/hi)
- Ключевая метрика — $CPI = mcycle/minstret$; задаёт базовую точку «до BPU»
- Неверные предсказания будущего BPU наблюдаемы как рост CPI при том же числе завершённых инструкций
- Реализовано в загрузочной прошивке `sc-bl` (FSBL): машинный режим, прямой доступ к CSR, готовый канал UART

Реализация: безопасное чтение 64-битных счётчиков

Половины 64-битного счётчика читаются двумя инструкциями — между ними младшее слово может переполниться. Старшее слово пересчитывается до и после:

```
#define read_counter64(dst, lo, hi) do{                               |\n    uint32_t _h, _l, _h2;                                           |\n    do { _h = read_csr(hi); _l = read_csr(lo);                       |\n        _h2 = read_csr(hi); } while (_h != _h2);                   |\n    (dst) = ((uint64_t)_h << 32) | _l; }while(0)
```

Оформлено как Pull Request в репозиторий прошивки sc-bl; включается флагом сборки PLF_HWCNT_DUMP.

В соответствии с поставленными задачами:

- выполнен обзор подходов к реализации BPU (статические схемы, динамические — от 2-битных счётчиков до gselect/gshare и многотабличных)
- проведён обзор открытых RISC-V-ядер (Ibex, CV32E40P, VexRiscv, CVA6) с точки зрения внедрения BPU
- предложены подходящие для scr1 направления (лёгкий статический либо минимальный bimodal/gshare)
- реализован дамп аппаратных счётчиков scr1 (PR в прошивку sc-bl)

Дальнейшие планы: снять профиль ветвлений реальных нагрузок, синтезировать предложенный предсказатель, эмпирически оценить выигрыш в сопоставлении с приростом площади и энергопотребления.